

Laboratorium – dokumentowe bazy danych: Couchbase

Od baz kolumnowych do
dokumentowych

JSON, indeksy, JOIN i
UNNEST



Co dziś robimy?

- Uruchamiamy i sprawdzamy środowisko Couchbase
- Poznajemy dane Northwind zapisane jako dokumenty JSON
- Sprawdzamy, dlaczego w Couchbase indeksy są obowiązkowe
- Łączymy dokumenty przez JOIN
- Porównujemy dwa modele: osobne kolekcje vs zagnieżdżony
- Używamy UNNEST do analizy tablic w dokumencie
- Czytamy plan zapytania przez EXPLAIN



Po tych zajęciach macie umieć

- wyjaśnić, czym Couchbase różni się od bazy relacyjnej
- wskazać strukturę bucket → scope → collection
- napisać podstawowe zapytania SQL++ / N1QL
- utworzyć i uzasadnić indeks celowy
- wykonać JOIN między kolekcjami dokumentów
- użyć UNNEST dla tablicy wewnątrz dokumentu
- porównać model niezagnieżdżony i zagnieżdżony
- powiedzieć, co widać w planie EXPLAIN



Dlaczego baza dokumentowa?

- W bazie relacyjnej dane są w tabelach: wiersz = rekord
- W bazie dokumentowej jednostką jest dokument JSON

```
{  
  "OrderID": 10248,  
  "CustomerID": "VINET",  
  "ShipCountry": "France"  
}
```

- Dokumenty nie muszą mieć identycznego zestawu pól
- Najważniejsza różnica: elastyczna struktura dokumentu



Couchbase w jednym zdaniu

- Baza dokumentowa oparta na JSON z językiem SQL++
- Składnia podobna do SQL, ale pracujemy na:
 - dokumentach JSON i kolekcjach
 - polach dokumentów i tablicach zagnieżdżonych
 - indeksach projektowanych pod zapytania
- Korzystamy z webowego klienta:
 - <http://localhost:8091>



Struktura danych w Couchbase

- Relacyjna

baza danych → schemat → tabela → wiersz → kolumna

- Couchbase

bucket → scope → collection → document → field

- W naszym laboratorium:

```
bucket: north0
scope: _default
collections:
  orders
  orderdetails
  customers
  products
  orders_nested
```



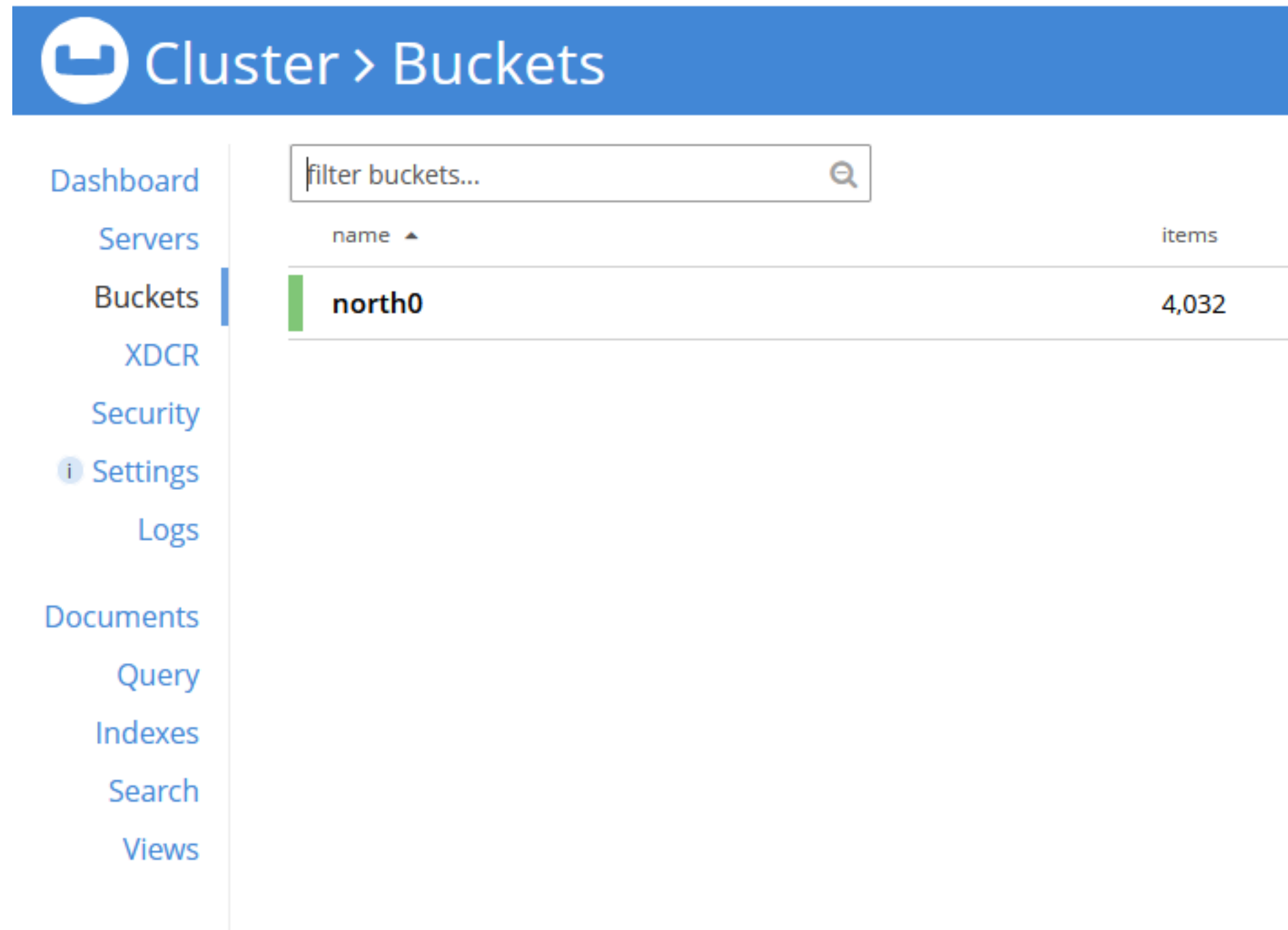
Dane Northwind

- orders – zamówienia (830 dokumentów)
- orderdetails – pozycje zamówień (2155)
- customers – klienci (91)
- products – produkty (77)
- orders_nested – zamówienia z tablicą pozycji items (830)



Couchbase UI – gdzie co jest?

- Buckets – podgląd bucketów, scope'ów i kolekcji



name ▲	items
north0	4,032



Couchbase UI – gdzie co jest?

- Query – edytor zapytań SQL++



The screenshot shows the Couchbase Query Editor interface. At the top, a blue header bar contains the Couchbase logo and the text "Cluster > Query". Below the header, a sidebar on the left lists navigation options: Dashboard, Servers, Buckets, XDCR, Security, Settings (with an 'i' icon), Logs, Documents, Query (highlighted with a blue bar), Indexes, Search, and Views. The main area is titled "Query Editor" and contains a SQL++ query:

```
1 SELECT COUNT(1) AS orders_nested_count
2 FROM `north0`._default.orders_nested
3 WHERE OrderID IS NOT MISSING;
```

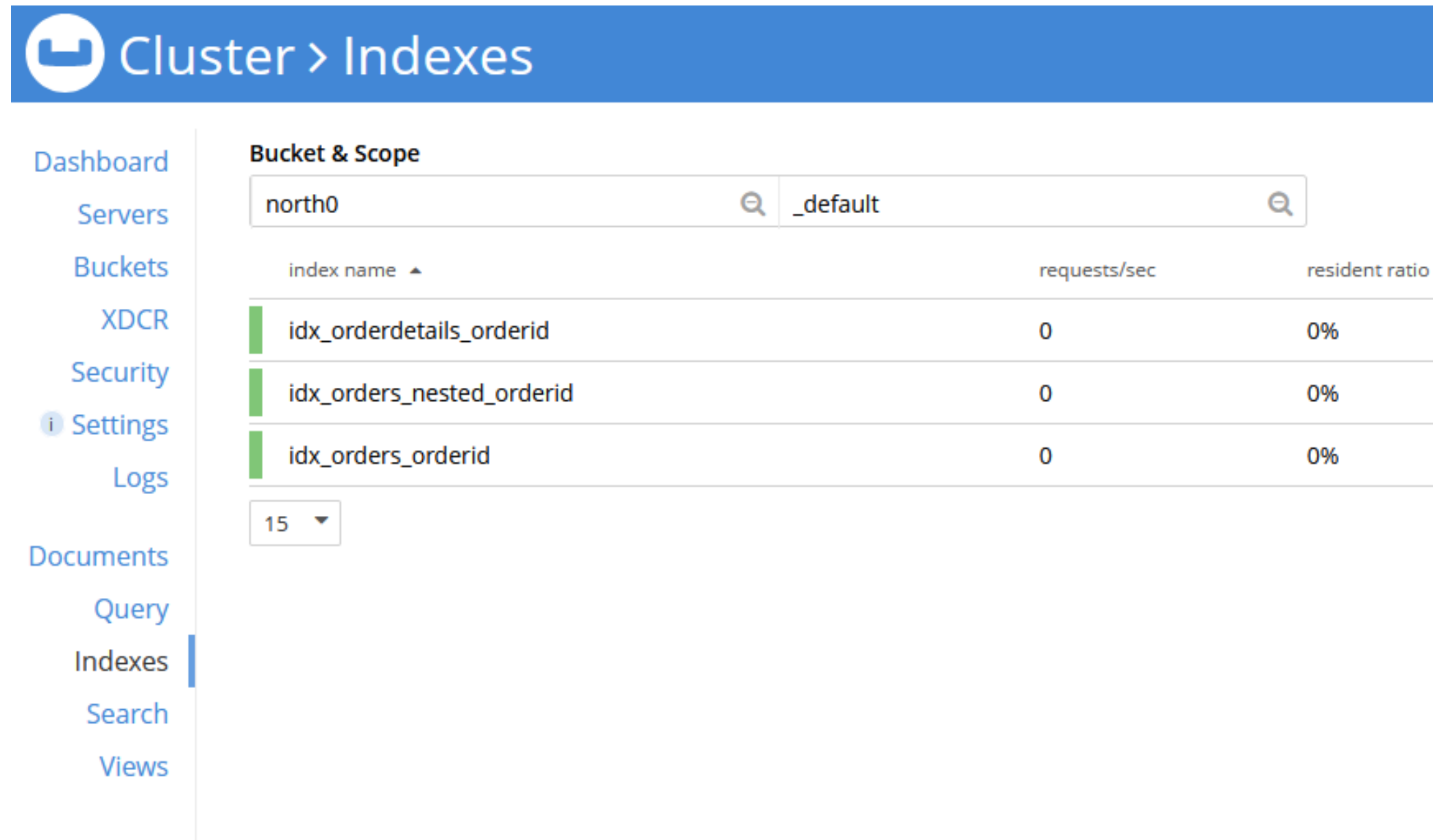
Below the query, there are buttons for "Execute" and "Explain", followed by the text "External Query Advisor", a green checkmark, "success", "Thu May 07 2026", and "1 docs". To the right of the query editor is a link for "history (32/33)". Below the query editor, the "Results" section shows a single document:

```
1 {"No data to display": "Hit execute to run query."}
```



Couchbase UI – gdzie co jest?

- Indexes – lista indeksów



The screenshot shows the Couchbase UI interface for the 'Cluster > Indexes' page. The left sidebar contains navigation links: Dashboard, Servers, Buckets, XDCR, Security, Settings (highlighted with an 'i' icon), Logs, Documents, Query, Indexes (highlighted with a blue bar), Search, and Views. The main content area has a blue header bar with the Couchbase logo and 'Cluster > Indexes'. Below this, there's a 'Bucket & Scope' section with two search boxes containing 'north0' and '_default'. A table lists the indexes with columns for 'index name', 'requests/sec', and 'resident ratio'. The table shows three indexes: 'idx_orderdetails_orderid', 'idx_orders_nested_orderid', and 'idx_orders_orderid', all with 0 requests/sec and 0% resident ratio. A dropdown menu at the bottom left of the table shows '15'.

index name ▲	requests/sec	resident ratio
idx_orderdetails_orderid	0	0%
idx_orders_nested_orderid	0	0%
idx_orders_orderid	0	0%



Dwa warianty modelowania danych

- Model niezagnieżdżony:
 - osobne kolekcje: orders, orderdetails, customers, products
 - aby połączyć dane → JOIN
- Model zagnieżdżony:
 - kolekcja orders_nested
 - pozycje zamówienia są w tablicy items → UNNEST



Model niezagnieżdżony: orders + orderdetails

- Przykład logiczny – dwie kolekcje:

- Aby zejść do pozycji zamówienia:

```
FROM orders AS o  
JOIN orderdetails AS od  
  ON od.OrderID = o.OrderID
```

```
orders  
  OrderID  
  CustomerID  
  OrderDate
```

```
orderdetails  
  OrderID  
  ProductID  
  UnitPrice  
  Quantity  
  Discount
```

- To przypomina klasyczny SQL



Model zagnieżdżony: orders_nested

- Przykład logiczny:

```
{
  "OrderID": 10248,
  "CustomerID": "VINET",
  "items": [
    {
      "ProductID": 42,
      "UnitPrice": 9.8,
      "Quantity": 10,
      "LineValue": 98
    }
  ]
}
```

- Pozycje są już w środku dokumentu
- Nie potrzebujemy JOIN potrzebujemy UNNEST



SQL++ / N1QL – wygląda znajomo

```
SELECT
  o.OrderID,
  o.CustomerID,
  o.OrderDate
FROM `north0`._default.orders AS o
WHERE o.OrderID IS NOT MISSING
LIMIT 10;
```

- Podobieństwa: SELECT, FROM, WHERE, GROUP BY, ORDER BY, JOIN, LIMIT



SQL++ / N1QL – wygląda znajomo

- Różnice:
 - ścieżka do kolekcji: ``north0`._default.orders`
 - pola mogą być MISSING
 - IFMISSINGORNULL(...) do braków danych
 - dokumenty mogą zawierać tablice i obiekty



Brak indeksu może zatrzymać zapytanie

- W Couchbase nie wystarczy napisać poprawnego zapytania
- Trzeba mieć jeszcze odpowiedni indeks

This query requires an index. A primary index is simple & quick to build (it is also resource-intensive and NOT recommended in production).

Create Primary Index

Learn more about Couchbase indexes

- Projektując zapytanie, projektujesz również indeks



Primary index

- Pozwala uruchomić zapytania **eksploracyjne**

```
CREATE PRIMARY INDEX idx_customers_primary  
ON `north0`._default.customers;
```

- Własności
 - Jest wygodny na start
 - Może do pełnego skanu danych
 - Mie jest rozwiązaniem produkcyjnym
 - Traktujemy jako „koło ratunkowe”



Indeks celowy

- Tworzymy pod konkretne pole i konkretne zapytanie

```
CREATE INDEX idx_customers_country  
ON `north0`._default.customers(Country);
```

- Pomaga plannerowi użyć indeksu
- Indeksy tworzy się pod zapytania, nie „na zapas”

```
SELECT  
  c.Country,  
  COUNT(1) AS customers_count  
FROM `north0`._default.customers AS c  
WHERE c.Country IS NOT MISSING  
GROUP BY c.Country  
ORDER BY customers_count DESC;
```


Braki danych: MISSING, NULL, IFMISSINGORNULL

- Pole w dokumencie JSON może:
 - istnieć i mieć wartość
 - istnieć z wartością NULL
 - nie istnieć czyli być MISSING
- W zadaniach może być potrzeba skorzystania z wzorca:

```
IFMISSINGORNULL(Discount, 0)
```

- Jeżeli rabatu nie ma traktuj do jako 0



JOIN w Couchbase

- JOIN łączy dokumenty z różnych kolekcji

```
SELECT
  a.pole1,
  a.pole2,
  b.pole3
FROM `north0`._default.kolekcjaA AS a
JOIN `north0`._default.kolekcjaB AS b
  ON a.klucz = b.klucz
WHERE a.klucz IS NOT MISSING
      AND b.klucz IS NOT MISSING
LIMIT 10;
```

- Relacja nie jest wymuszana jak w bazie relacyjnej



UNNEST: rozbijanie tablicy w dokumencie

- Dane są tablicą wewnątrz dokumentu, używamy UNNEST

```
SELECT
  alias.pole1,
  alias.pole2,
  element.pole_a,
  element.pole_b
FROM `north0`._default.kolekcja AS alias
UNNEST alias.tablica AS element
WHERE alias.pole_kluczowe IS NOT MISSING;
```

- Efekt: jeden dokument tworzy wiele rekordów z elementów tablicy



JOIN vs UNNEST

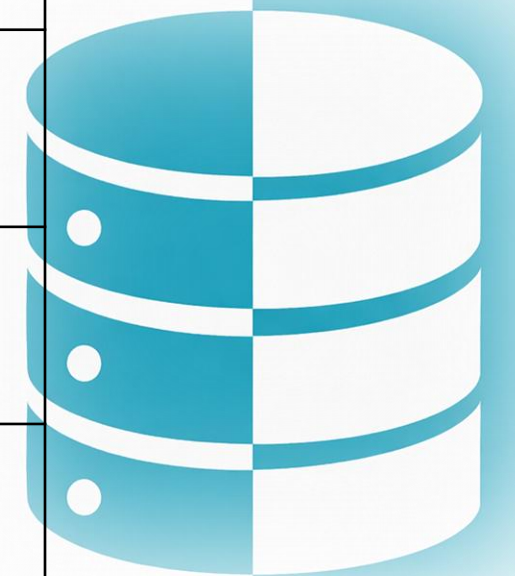
Sytuacja	Używamy
dane są w osobnych kolekcjach	`JOIN`
dane są w tablicy wewnątrz dokumentu	`UNNEST`

- Najważniejsze pytanie projektowe:
 - Czy pozycje zamówienia powinny być osobnymi dokumentami, czy częścią dokumentu zamówienia?
 - Nie ma jednej odpowiedzi – model zależy od użycia danych



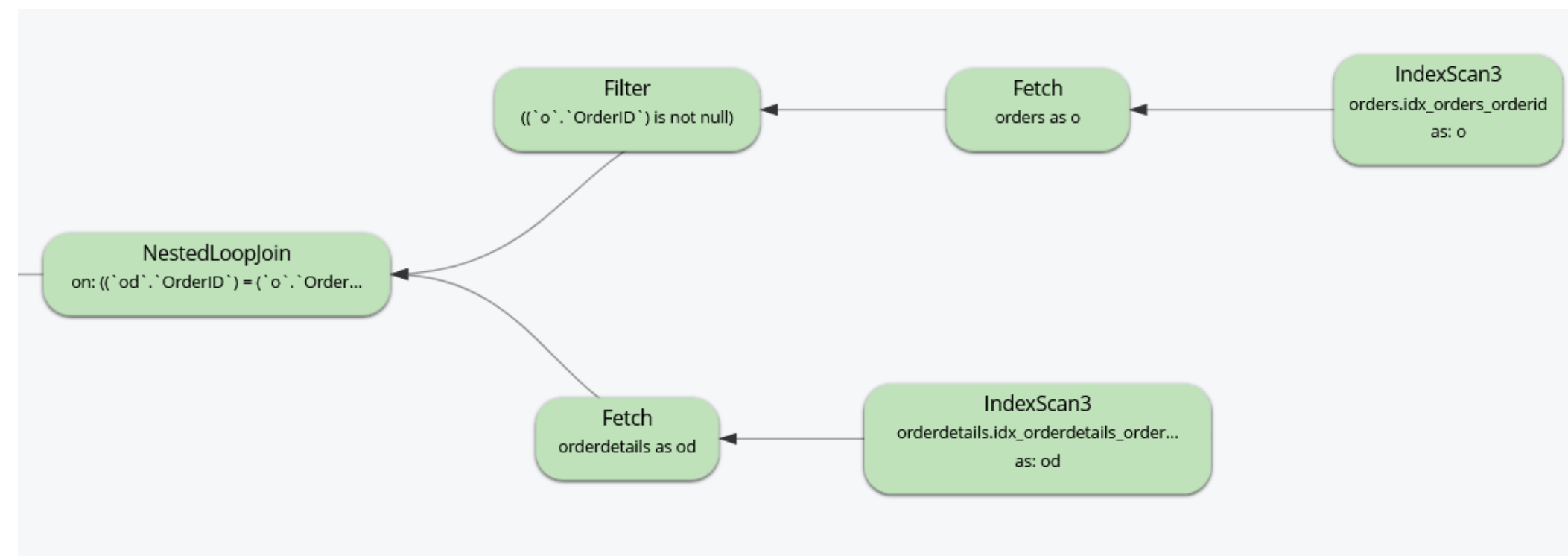
JOIN vs UNNEST

Pytanie projektowe	Lepsze podejście
Czy pozycje zamówienia istnieją tylko jako część zamówienia?	UNNEST / tablica w dokumencie
Czy zwykle pobieramy całe zamówienie razem z pozycjami?	UNNEST / tablica w dokumencie
Czy pozycji w zamówieniu jest niewiele i ich liczba jest przewidywalna?	UNNEST / tablica w dokumencie
Czy pozycje mają własny cykl życia, statusy, aktualizacje, historię?	JOIN / osobne dokumenty
Czy często analizujemy pozycje niezależnie od zamówień?	JOIN / osobne dokumenty
Czy liczba pozycji może być bardzo duża albo nieograniczona?	JOIN / osobne dokumenty
Czy wiele procesów może równocześnie modyfikować różne pozycje?	JOIN / osobne dokumenty



EXPLAIN

- EXPLAIN pokazuje plan wykonania zapytania
- Najważniejsze elementy planu:
 - IndexScan – użycie indeksu
 - PrimaryScan – skan przez primary index
 - Fetch – pobranie pełnego dokumentu
 - NestedLoopJoin – wykonanie JOIN
 - Unnest – rozbicie tablicy
 - Group / Order / Limit



Jak czytać plan w uproszczeniu?

- Nie musicie rozumieć każdego szczegółu
- Wystarczy odpowiedzieć:
 - Czy zapytanie korzysta z indeksu?
 - PrimaryScan czy IndexScan?
 - Czy baza wykonuje JOIN?
 - Czy pojawia się Unnest?
 - Czy jest agregacja, sortowanie i limit?



Zadania – ogólny plan

Zadanie	Temat	Punkty
0	Gotowość środowiska	0
1	Pierwsze poznanie Couchbase i danych	1
2	Indeksy: brak indeksu, primary index, secondary index	2
3	JOIN na kolekcjach dokumentów	2
4	Model niezagnieżdżony vs zagnieżdżony: JOIN vs UNNEST	2
5	Agregacja biznesowa	1
6	EXPLAIN i refleksja końcowa	2
Razem		10



Zadanie 0: gotowość środowiska

- Docker działa:
- Couchbase UI pod localhost:8091
- logowanie student / student
- bucket north0 widoczny
- SELECT 1 AS test działa w Query Workbench
- Bez punktów, ale konieczne do dalszej pracy



Zadanie 1: poznanie danych

- Jakie kolekcje są dostępne?
 - Ile dokumentów w każdej kolekcji?
 - Jak wygląda dokument w orders?
 - Jak wygląda dokument w orders_nested?
-
- W komentarzu:
 - Czym różni się zamówienie od zamówienia zagnieżdżonego?



Zadanie 2: indeksy

- Trzy sytuacje:
 - zapytanie bez indeksu
 - po primary index
 - po indeksie celowym
- Najważniejszy wniosek:
 - Indeks jest częścią projektowania zapytania



Zadanie 3: JOIN

- Łączycie: orders + customers oraz orders + orderdetails
- Liczycie wartość zamówienia

```
UnitPrice * Quantity * (1 - Discount)
```

- W komentarzu:
 - składnia jak SQL?
 - po czym łączone?
 - dlaczego indeksy?
 - największe zamówienia = najwięcej pozycji?



Zadanie 4: JOIN vs UNNEST

- Porównujecie dwa podejścia

Podejście 1

```
orders + orderdetails + JOIN
```

Podejście 2

```
orders_nested + UNNEST items
```

- Sprawdzacie:
 - czy wyniki zgodne
 - czym różni się model danych
 - kiedy zagnieżdżenie może być wygodne



Zadanie 5: agregacja biznesowa

- Wybieracie jeden wariant:
 - A: top 10 produktów po wartości sprzedaży
 - B: top 10 klientów po wartości zakupów
- Couchbase nadaje się też do analiz biznesowych?



Zadanie 6: EXPLAIN i refleksja

- Uruchamiacie EXPLAIN dla:
 - jednego zapytania z JOIN
 - jednego zapytania z UNNEST



Powodzenia!